Note de séance SQL n°5

Fonctions de fenêtrage | 15 et 16 octobre 2026

1. L'idée de la séance

Avec **GROUP BY**, les lignes sont regroupées et le détail disparaît. Une **fonction de fenêtrage** fait un calcul sur un groupe de lignes **sans les regrouper** : chaque ligne reste, et le résultat s'ajoute dans une nouvelle colonne. On la reconnaît au mot-clé `OVER`. `PARTITION BY` découpe le tableau en fenêtres, `ORDER BY` (dans OVER) fixe l'ordre du calcul.

2. Les fonctions vues

Fonction	Rôle	Remarque
ROW_NUMBER()	Numéroter les lignes (1, 2, 3...)	Numéros tous différents, même en cas d'égalité
RANK()	Classer les lignes	Ex æquo : même rang (1, 1, 3)
SUM / AVG ... OVER	Agrégation en fenêtre	OVER () = toutes les lignes
SUM ... OVER (ORDER BY)	Cumul	Le total s'additionne ligne après ligne
LAG(col)	Valeur de la ligne précédente	NULL sur la première ligne
LEAD(col)	Valeur de la ligne suivante	NULL sur la dernière ligne
NTILE(n)	Découper en n tranches égales	Utile pour les quantiles

3. Exercices et corrections

```
-- Numéroter les commandes de la plus chère à la moins chère
SELECT *, ROW_NUMBER() OVER (ORDER BY montant DESC) AS numero
FROM commandes;

-- Ecart entre l'âge d'un client et l'âge moyen de sa ville
-- Kinshasa (31.5) : Awa -3.5, Marie -9.5, Grace -0.5, Eric +13.5
SELECT prenom, age, ville,
       age - AVG(age) OVER (PARTITION BY ville) AS difference
FROM clients;

-- Classement par âge croissant dans chaque ville
SELECT *, RANK() OVER (PARTITION BY ville ORDER BY age) AS numero_age
FROM clients;

-- Cumul des montants, de la fin vers le début (dernière valeur : 1555)
SELECT *, SUM(montant) OVER (ORDER BY id DESC) AS cumul
FROM commandes
ORDER BY id DESC;
```

4. Mini-défis et corrections

```
-- Le client le plus âgé de chaque ville : Eric, Sarah, Patrick
WITH ageville AS (
    SELECT id, prenom, age, ville,
           ROW_NUMBER() OVER (PARTITION BY ville ORDER BY age DESC) AS rang
```

```
    FROM clients
)
SELECT * FROM ageville WHERE rang = 1;

-- Pourcentage de chaque commande dans le total (1555) : Ordinateur 51.45
SELECT *, ROUND(montant * 100.0 / SUM(montant) OVER (), 2) AS pourcentage
FROM commandes;

-- Commande précédente et différence
WITH com AS (
    SELECT *, LAG(montant) OVER (ORDER BY id) AS montprec
    FROM commandes
)
SELECT *, montant - montprec AS difference
FROM com;

-- LEAD : la commande suivante
SELECT id, montant, LEAD(montant) OVER (ORDER BY id) AS montant_suivant
FROM commandes;

-- NTILE : 3 tranches de la plus chère à la moins chère (3 / 2 / 2 lignes)
SELECT id, montant, NTILE(3) OVER (ORDER BY montant DESC) AS tranche
FROM commandes;
```

5. À retenir

• On ne peut **pas** filtrer sur une fonction de fenêtrage dans un WHERE : on la calcule dans un **CTE**, puis on filtre dans la requête finale.

• **ROW_NUMBER** vs **RANK** : en cas d'égalité, ROW_NUMBER n'en garde qu'une au hasard, RANK les garde toutes. Choisir selon la question.

• Le ORDER BY placé dans OVER définit l'ordre du calcul, pas forcément celui de l'affichage : ajouter un ORDER BY à la fin de la requête.

• Division entière : sur PostgreSQL ou SQL Server, `montant * 100 / SUM(...)` tronque le résultat. La version portable utilise `100.0`.

• `LAG(montant, 1, 0)` remplace le NULL de la première ligne par 0.

• GROUP BY réduit le nombre de lignes, une fonction de fenêtrage les conserve toutes.

6. Prochaine séance

CASE WHEN, fonctions de dates (YEAR, MONTH, DATE_FORMAT, DATEDIFF) et fonctions de texte (LIKE, CONCAT, UPPER, TRIM, SUBSTRING).